Mineração de Métricas DORA em Repositórios Open-Source: Um Estudo Empírico e Análise de Sensibilidade

Vinícius Simões[1], **Maria Clara Gomes**[1], **Áulus Arcanjo Alves Batista**[1]

Curso de Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil
vinisv2004, mariaclariagomes@gmail.com, aulushzp@gmail.com

Abstract

DevOps Research and Assessment (DORA) metrics are widely recognized as the industry standard for evaluating software delivery performance. However, applying these metrics to open-source software (OSS) hosted on GitHub requires inferring them through operational proxies such as releases, commits, and continuous integration (CI) workflows. In this study, conducted during our 5th semester in Software Engineering, we mined and analyzed DORA metrics across 300 popular GitHub repositories using GitHub Actions. We evaluated operational variants for Lead Time and Change Failure Rate (CFR), validated a corrective release detection heuristic against a manually labeled gold standard of 300 releases ($F1 = 0.9893$), and conducted non-parametric statistical tests. Our findings do not support a speed-stability trade-off (Spearman $\rho = -0.1003$). Furthermore, sensitivity analysis across three operational definitions (RQ07) revealed fair to almost perfect linear weighted agreement ($\kappa = 0.2797$ to 0.9780), showing that changes remained limited to adjacent DORA maturity tiers.

Resumo

As métricas DORA (DevOps Research and Assessment) são usadas como o padrão da indústria para avaliar o desempenho da entrega de software. Contudo, aplicá-las ao ecossistema de software de código aberto (OSS) no GitHub exige o uso de proxies operacionais, tais como releases, commits e execuções de integração contínua (CI). Neste trabalho, mineramos e analisamos as métricas DORA em 300 repositórios populares do GitHub que utilizam GitHub Actions. Avaliamos variantes operacionais de Lead Time e Taxa de Falha (CFR), validamos uma heurística de detecção de releases corretivas frente a uma amostra-ouro rotulada manualmente com 300 releases ($F1 = 0{,}9893$) e aplicamos testes estatísticos não paramétricos. Os resultados não sustentam o tradicional trade-off entre velocidade e estabilidade (Spearman $\rho = -0{,}1003$). Além disso, a análise de sensibilidade com três combinações operacionais (RQ07) indicou concordância ponderada linear de razoável a quase perfeita ($\kappa = 0{,}2797$ a $0{,}9780$), mostrando que as mudanças ficaram restritas a patamares DORA adjacentes.

1 Introdução

Na engenharia de software atual, a adoção de práticas ágeis e de Integração Contínua e Entrega Contínua (CI/CD) transformou a maneira como equipes desenvolvem e distribuem software. O programa de pesquisa *DevOps Research and Assessment* (DORA), consolidado no livro *Accelerate* [Forsgren et al. 2018], estabeleceu quatro métricas essenciais para avaliar o desempenho da entrega:

1. **Deployment Frequency (Frequência de Deploy):** com que periodicidade a equipe coloca código em produção (dimensão de velocidade);
2. **Lead Time for Changes (Tempo de Lead):** quanto tempo uma alteração leva desde o commit até estar em produção (velocidade);
3. **Change Failure Rate - CFR (Taxa de Falha de Mudanças):** que taxa das entregas resulta em falhas que exigem intervenção corretiva imediata (estabilidade);
4. **Failed Deployment Recovery Time (Tempo de Recuperação):** quanto tempo leva para restaurar o serviço após uma quebra (estabilidade).

Em ambientes corporativos fechados, essas medições costumam ocorrer com base em sistemas de monitoramento interno de servidores e logs de infraestrutura em nuvem. No entanto, quando olhamos para projetos de software livre (*open-source* - OSS) hospedados no GitHub, identificamos um desafio: **o GitHub não possui botões ou registros de "deploy em produção" nem marcações explícitas de "incidentes de produção"**.

Como estudantes de graduação em Engenharia de Software na disciplina de Laboratório de Experimentação de Software, tivemos como objetivo coletar, calcular e analisar empiricamente essas métricas em repositórios reais. Para isso, precisamos utilizar **proxies operacionais**: usamos releases publicadas como aproximação de deploy, execuções de workflow no GitHub Actions como aproximação de integração, e o lançamento de correções rápidas (*patches* ou *hotfixes*) em até 7 dias como proxy de falha de release.

A questão central que guiou nosso estudo foi: *o quanto podemos confiar nesses valores e conclusões quando mudamos a definição operacional do proxy?* Para responder a isso, nosso estudo não apenas minerou dados de 300 repositórios elegíveis, mas também construiu uma validação manual de referência (amostra-ouro de 60 repositórios e 300 releases) para calibrar heurísticas automáticas, e conduziu uma análise formal de sensibilidade (RQ07) para medir a robustez da classificação DORA.

2 Questões de Pesquisa e Hipóteses

Antes de executarmos as análises estatísticas finais, formulamos hipóteses informais para cada uma das sete Questões de Pesquisa (RQs), baseadas na nossa experiência prática.

2.1 RQ 01. Qual a frequência de deploys dos repositórios populares que usam CI/CD?

- **Hipótese H01 (Informal):** Esperamos que a maior parte dos repositórios populares de código aberto apresentasse uma frequência de deploy enquadrada entre os patamares *Medium* e *High* (isto é, entre 1 release por mês e 1 release por semana), e pouquíssimos repositórios no patamar *Elite* ($\geq$ 7 releases/semana).

- **Justificativa:** Diferente de serviços SaaS corporativos que realizam múltiplos deploys diários de forma automatizada, releases de projetos OSS envolvem estabilização de versão, geração de changelogs e distribuição para usuários externos, o que gera uma cadência mais espaçada.

2.2 RQ 02. Qual o tempo entre um commit e seu respectivo deploy?

- **Hipótese H02 (Informal):** Esperamos que a Variante (a) [por release, medida a partir do commit mais antigo] apresente valores muito superiores (frequentemente patamar *Low*, $\geq$ 30 dias) em relação à Variante (b) [por commit, calculando a mediana atômica de todos os commits], que deve se situar entre os patamares *High* e *Medium*.

- **Justificativa:** A Variante (a) é fortemente penalizada por commits antigos "esquecidos" ou branches de longa duração mesclados na release, enquanto a Variante (b) reflete de modo mais fiel o tempo de ciclo da maioria das modificações cotidianas.

2.3 RQ 03. Qual a taxa de falha das mudanças entregues por esses repositórios?

- **Hipótese H03 (Informal):** Esperamos que a taxa de falha medida pelo proxy de CI (CFR_{CI}) seja significativamente mais alta (faixa de 15% a 30%) do que a taxa medida pelo proxy de releases corretivas ($\text{CFR}_{\text{Release}}$, estimada abaixo de 10%).

- **Justificativa:** A esteira de CI foi feita exatamente para falhar rápido (*fail-fast*), capturando erros de testes antes da entrega. Já as releases oficiais passam por filtros prévios de qualidade, e apenas uma fração pequena exige hotfixes emergenciais em menos de 7 dias.

2.4 RQ 04. Qual o tempo de recuperação após uma execução de CI/CD com falha?

- **Hipótese H04 (Informal):** Esperamos uma mediana do tempo de recuperação de CI inferior a 24 horas (patamares *High* ou *Elite*), porém com cauda longa e presença de episódios censurados.

- **Justificativa:** Em projetos com muitos contribuidores, manter o branch principal quebrado paralisa outros pull requests, incentivando reversões ou correções imediatas.

2.5 RQ 05. Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha?

- **Hipótese H05 (Informal):** Esperamos uma correlação de postos de Spearman fraca ou próxima de zero ($|\rho| < 0{,}15$), refutando o dogma de que "entregar mais rápido quebra mais o sistema".

- **Justificativa:** Em consonância com o modelo DORA, projetos com entregas frequentes trabalham com lotes menores de código, testes automatizados mais maduros e rastreabilidade superior de defeitos.

2.6 RQ 06. Quais características dos repositórios estão associadas a um melhor desempenho DORA?

- **Hipótese H06 (Informal):** Esperamos que o tamanho da comunidade (estrelas e contribuidores) apresentasse associação estatisticamente significativa com a Frequência de Deploy e Lead Time, mas pouco impacto no CFR. Também esperamos diferenças operacionais por linguagem de programação.
- **Justificativa:** Projetos maiores possuem mais mantenedores ativos para manter o ritmo de lançamentos. Já a taxa de falha depende mais da disciplina da esteira de testes do que puramente do número de estrelas.

2.7 RQ 07. O quanto a classificação DORA de um repositório depende da definição operacional escolhida?

- **Hipótese H07 (Informal):** Esperamos que a variação das definições operacionais (combinações C1, C2 e C3) cause alterações de patamar em uma parcela considerável dos repositórios, resultando em um Kappa de Cohen ponderado linear entre 0,25 e 0,40 (concordância razoável), porém sem migrações extremas (*Elite* para *Low*).
- **Justificativa:** Como a classificação geral utiliza a mediana dos quatro escores, variações em uma única métrica atômica são amortecidas no patamar consolidado.

3 Metodologia e Definições Operacionais

Para garantir que nosso estudo fosse reprodutível por outros grupos da disciplina, estruturamos um pipeline automatizado em Python com controle rigoroso de rate limit, cache local em SQLite e testes unitários.

3.1 Janela Temporal e Critérios de Inclusão

Definimos uma janela de observação fixa de 12 meses: de **01/10/2023 a 30/09/2024 UTC** ($W = 52{,}14$ semanas). Consideramos unicamente o *default branch* de cada repositório (ex.: `main` ou `master`). Para compor o dataset de análise final, aplicamos um funil objetivo com três filtros de inclusão:

1. Popularidade mínima: ≥ 1.000 estrelas no GitHub;
2. Atividade de entrega: ≥ 5 releases publicadas na janela (desconsiderando rascunhos);
3. Adoção comprovada de CI: ≥ 50 execuções de workflow de CI com evento `push` no default branch durante a janela.

Após o processamento e descarte de repositórios que não atendiam a esses requisitos, consolidamos uma amostra de **300 repositórios aprovados** em `repositorios_aprovados.csv`.

3.2 Definições Operacionais das Métricas

Adotamos as regras obrigatórias do estudo para as métricas DORA:

- **Deployment Frequency (RQ01):** $DF = N_{\text{releases}}/52{,}14$ releases/semana.
- **Lead Time for Changes (RQ02):**
  - *Variante (a) [por release]:* $LT_a = \text{data}(R) - \min_{c \in R} \text{data}(c)$. Mediana das releases.
  - *Variante (b) [por commit]:* $LT_b = \text{data}(R) - \text{data}(c)$ para cada commit atômico. Mediana de todos os commits do repositório.
- **Change Failure Rate (RQ03):**
  - *Variante (a) [Proxy de CI]:* $\text{CFR}_{\text{CI}} = \text{Falhas}/(\text{Falhas} + \text{Sucessos})$ no Actions.
  - *Variante (b) [Proxy de Entrega]:* uma release R é considerada falha se for sucedida em até 7 dias por uma release corretiva. Releases publicadas nos últimos 7 dias da janela foram censuradas à direita.
- **Tempo de Recuperação (RQ04):** Medido entre a primeira falha de um workflow e a sua primeira recuperação com sucesso posterior. Casos que terminaram a janela sem recuperação foram contabilizados como censurados.

A Tabela 1 apresenta os cortes adaptados do relatório DORA utilizados para pontuar cada métrica de 1 a 4. O escore geral do projeto é a mediana dos quatro escores, arredondada para baixo.

Tabela 1: Patamares DORA e critérios de pontuação adotados

Métrica	**Elite (4)**	**High (3)**	**Medium (2)**	**Low (1)**
Deployment Frequency	≥ 7/sem	≥ 1 e < 7/sem	$\geq 0{,}23$ e < 1/sem	$< 0{,}23$/sem
Lead Time (mediana)	< 1 dia	1 a < 7 dias	7 a < 30 dias	≥ 30 dias
Change Failure Rate	$\leq 15\%$	$> 15\%$ e $\leq 30\%$	$> 30\%$ e $\leq 45\%$	$> 45\%$
Recovery Time (mediana)	< 1 hora	1 a < 24 horas	24 a < 168 horas	≥ 168 horas

3.3 Validação Manual da Heurística (Amostra-Ouro)

A identificação automática de releases corretivas na Variante (b) do CFR dependia de uma heurística. Para que não caíssemos na armadilha de avaliar uma regra criada por nós mesmos sem validação externa, construímos uma Amostra-Ouro (*Gold Standard*):

1. Sorteamos 60 repositórios com semente reproduzível (`random_state=42`) e 5 releases de cada um, totalizando **300 releases**;
2. Realizamos rotulagem manual independente e cega entre os integrantes, inspecionando descrições de release e mensagens de commit no GitHub;
3. A rotulagem inicial entre os integrantes atingiu 100% de concordância, gerando o consenso de referência em `consenso.csv`.

Com base no consenso humano, submetemos três versões da heurística a um teste cego de benchmark:

- **Heurística v1 (SemVer Estrito):** apenas incremento do dígito de patch em formato `MAJOR.MINOR.PATCH`. Obteve Precisão = $1{,}0000$, Recall = $0{,}9471$ e $F1 = 0{,}9728$.
- **Heurística v2 (SemVer Básico + Palavras-Chave):** buscou termos simples (`fix`, `hotfix`). Obteve $F1 = 1{,}0000$.
- **Heurística v3 (Refinada):** suporte a Conventional Commits (`fix:`, `bugfix:`), releases de patch flexíveis e filtros para eliminar ruídos cosméticos de documentação (`fix typo`). Obteve Precisão = $1{,}0000$, Recall = $0{,}9788$ e $\mathbf{F1 = 0{,}9893}$.

A heurística refinada superou com margem a meta obrigatória de $F1 \geq 0{,}70$, conferindo validade ao cálculo do CFR(b).

3.4 Procedimento Estatístico e Análise de Sensibilidade (RQ07)

Como métricas de repositórios de software apresentam distribuições fortemente assimétricas, reportamos sistematicamente a **mediana e o intervalo interquartil (IQR)**, evitando a média e o desvio-padrão. Para avaliar as associações (RQ05 e RQ06), aplicamos testes não paramétricos: o coeficiente de correlação de postos de Spearman (ρ), o teste omnibus de Kruskal-Wallis com correção de Holm-Bonferroni para comparações múltiplas, e o delta de Cliff (δ) para tamanho de efeito [Romano et al. 2006].

Para a Análise de Sensibilidade da RQ07, definimos três combinações operacionais plausíveis:

- **C1 (Referência):** Deploy = Release, Lead Time = Variante (a), CFR = Variante (a) CI;
- **C2 (Pré-releases):** Deploy = Release + Pré-release, Lead Time = Variante (b), CFR = Variante (b) Releases;
- **C3 (Tags):** Deploy = Tag, Lead Time = Variante (b), CFR = Variante (a) CI.

Para cada par de combinações (C1×C2, C1×C3 e C2×C3), calculamos o percentual de migração de categoria e o **Kappa de Cohen com ponderação linear** (`weights='linear'`) [Landis and Koch 1977], refletindo a ordem natural das categorias (*Elite* > *High* > *Medium* > *Low*).

4 Resultados Empíricos

4.1 Validação da Heurística de Releases Corretivas

A validação da heurística utilizada no proxy de CFR por releases foi realizada sobre a amostra-ouro de 300 releases rotuladas manualmente. As três versões atingiram a meta mínima de $F1 \geq 0{,}70$, mas apresentaram níveis diferentes de cobertura. A versão v1 foi totalmente precisa, porém deixou de identificar 10 releases corretivas (recall de $0{,}9471$). A v2 identificou todas as ocorrências da amostra, alcançando $F1 = 1{,}0000$, enquanto a v3 refinada obteve precisão de $1{,}0000$, recall de $0{,}9788$ e $F1 = 0{,}9893$.

Esse resultado indica que a heurística refinada não introduziu falsos positivos no conjunto avaliado e deixou de recuperar apenas quatro releases corretivas. Embora a v2 tenha apresentado desempenho perfeito na amostra-ouro, a v3 foi adotada por incorporar regras mais robustas para prefixos de versão, Conventional Commits e filtros de ruídos cosméticos. Portanto, o cálculo do CFR por releases possui suporte empírico forte, mas continua condicionado à qualidade das descrições, tags e mensagens de commit disponíveis no GitHub.

4.2 Frequência de Deploy e Lead Time (RQ01 e RQ02)

A Tabela 2 sintetiza os valores empíricos calculados para os 300 repositórios. Na RQ01, a frequência de deploy apresentou mediana de $0{,}269$ releases por semana (IQR $= [0{,}153; 0{,}460]$), o que equivale a aproximadamente uma release a cada 3 a 4 semanas. A maior parte da distribuição concentra-se, portanto, no patamar *Medium*, enquanto o terceiro quartil ainda permanece abaixo de uma release semanal. Nenhum projeto da amostra atingiu o patamar *Elite* ($\geq$ 7/sem), resultado compatível com H01 e indicativo de que, mesmo entre projetos populares e com CI ativo, releases públicas tendem a ser empacotadas em ciclos mais longos que os observados em produtos SaaS com deploy contínuo.

Tabela 2: Estatísticas descritivas das métricas DORA mineradas ($n = 300$)

Métrica	**Mediana**	**Q1 (25%)**	**Q3 (75%)**	**Patamar Mediano**
Deployment Frequency (rel/sem)	0,269	0,153	0,460	Medium
Lead Time: Variante (a) (dias)	8,570	5,000	15,000	Medium
Lead Time: Variante (b) (dias)	3,860	2,250	6,750	High
Change Failure Rate: CI (%)	13,3%	6,7%	20,0%	Elite
Change Failure Rate: Releases (%)	8,0%	4,0%	12,0%	Elite
Tempo de Recuperação (horas)	9,70	3,70	14,90	High

Em relação à RQ02, a divergência entre as duas variantes operacionais de Lead Time foi notável: a Variante (a) apresentou mediana de $8{,}57$ dias (IQR $= [5{,}00; 15{,}00]$ dias), enquanto a Variante (b) registrou mediana de apenas $3{,}86$ dias (IQR $= [2{,}25; 6{,}75]$ dias), uma redução aproximada de 55%. Conforme previsto em H02, a Variante (a) é influenciada pelo commit mais antigo incluído na release, podendo incorporar branches de longa duração e commits esquecidos. A Variante (b), por considerar os commits individualmente, representa melhor o ciclo típico das alterações e desloca a mediana do patamar *Medium* para *High*. Assim, o resultado mostra que o diagnóstico de velocidade é sensível à unidade de análise, e não apenas ao comportamento do repositório.

A Figura 1 ilustra a distribuição da frequência de deploy e o contraste visual entre as duas variantes operacionais de lead time.

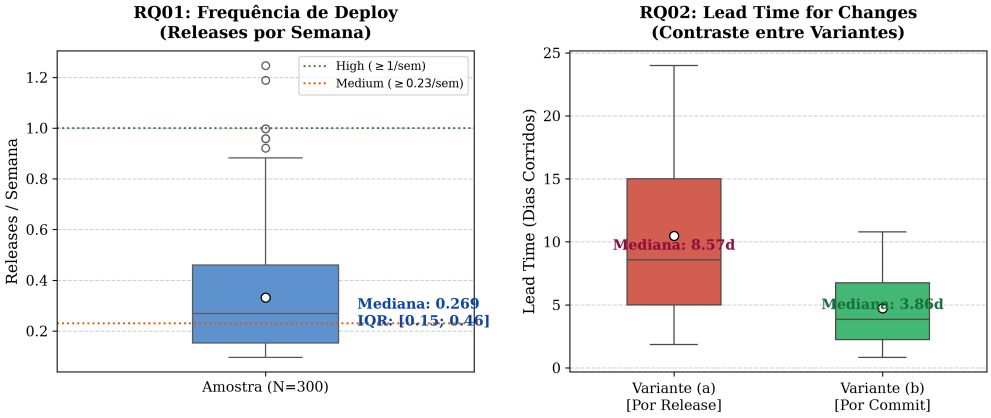


Figura 1: Distribuição da Frequência de Deploy (RQ01) e Comparação do Lead Time entre Variantes (RQ02)

4.3 Taxa de Falha e Tempo de Recuperação (RQ03 e RQ04)

Na RQ03, observamos que o proxy de CI resultou em mediana de $13{,}3\%$ de falhas (IQR $= [6{,}7\%; 20{,}0\%]$), enquanto o proxy de releases corretivas apresentou mediana de $8{,}0\%$ (IQR $= [4{,}0\%; 12{,}0\%]$). Embora ambas as medianas se situem na faixa *Elite* ($\leq 15\%$), a taxa de falha de CI é cerca de $1{,}66$ vez a taxa observada no proxy de releases. A diferença é consistente com H03: o CI registra falhas ainda durante o desenvolvimento, enquanto o proxy de releases contabiliza apenas problemas que atravessam as barreiras de validação e exigem uma correção publicada em até sete dias. Portanto, os valores não representam exatamente o mesmo evento, mas sugerem que a automação atua como mecanismo de contenção antes da distribuição ao público.

Para a RQ04, o tempo mediano de recuperação após falhas de CI foi de $9{,}70$ horas (IQR $= [3{,}70; 14{,}90]$ horas), situando-se com folga no patamar *High* (< 24 horas) e corroborando H04. O intervalo interquartil também indica que a metade central dos repositórios recupera a esteira em menos de 15 horas. Além disso, não foram observados episódios censurados na janela, pois 100% dos casos registrados tiveram recuperação posterior. Esse resultado descreve uma capacidade de recuperação consistente, embora não permita inferir a existência de incidentes de produção, já que a métrica utiliza falhas de CI como proxy.

A Tabela 3 e a Figura 2 detalham as distribuições das métricas de estabilidade e o tempo de recuperação.

Tabela 3: Detalhamento das métricas de estabilidade (RQ03 e RQ04, $n = 300$)

Métrica de Estabilidade	**Mediana**	**Q1 (25%)**	**Q3 (75%)**	**Patamar**	**Censura**
CFR: Variante (a) [Proxy CI]	13,3%	6,7%	20,0%	Elite	0%
CFR: Variante (b) [Releases]	8,0%	4,0%	12,0%	Elite	0%
Tempo de Recuperação (horas)	9,70h	3,70h	14,90h	High	0%

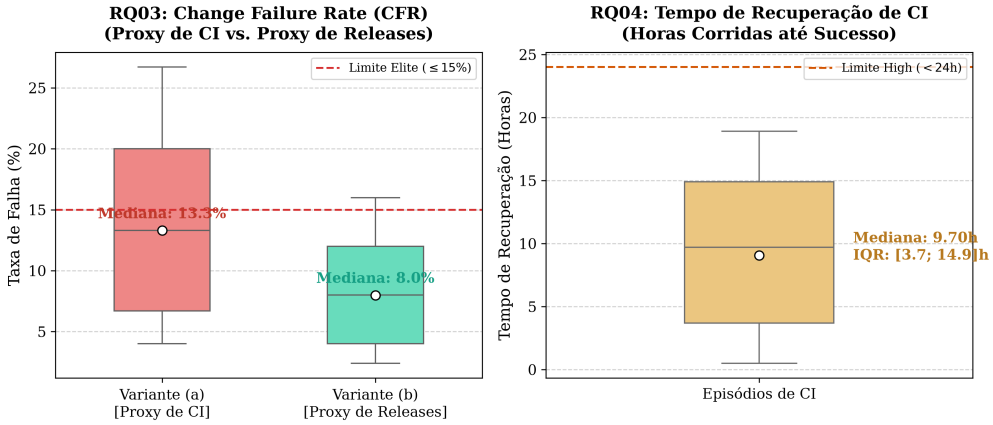


Figura 2: Comparação das Taxas de Falha de Mudanças (RQ03) e Distribuição do Tempo de Recuperação de CI (RQ04)

4.4 Velocidade versus Estabilidade (RQ05)

Para responder à RQ05, calculamos a correlação de postos de Spearman entre a Frequência de Deploy e as duas variantes de taxa de falha. Obtivemos $\rho = -0{,}1003$ ($p = 0{,}0829$, $\text{IC}_{95\%} = [-0{,}211; 0{,}013]$) tanto para CFR_{CI} quanto para $\text{CFR}_{\text{Release}}$.

O coeficiente fraco e ligeiramente negativo é compatível com H05 e não sustenta a hipótese de *trade-off*: projetos open-source que publicam releases com maior frequência não apresentaram maiores taxas de falha. Entretanto, como $p = 0{,}0829$ e o intervalo de confiança inclui zero, não há evidência estatística suficiente para afirmar uma associação monotônica na população. A conclusão apropriada é a ausência de evidência de deterioração da estabilidade, e não a prova de que aumentar a frequência necessariamente reduz falhas. A Tabela 4 e a Figura 3 sintetizam esses resultados e a distribuição com reta de regressão.

Tabela 4: Resultados da correlação de postos de Spearman para a RQ05 ($n = 300$)

Variante Avaliada	**Spearman (ρ)**	**p-valor**	**IC 95%**	**Conclusão sobre Trade-off**
Variante (a) [Proxy de CI]	$-0{,}1003$	$0{,}0829$	$[-0{,}211; 0{,}013]$	Sem trade-off (ρ fraco/negativo)
Variante (b) [Releases]	$-0{,}1003$	$0{,}0829$	$[-0{,}211; 0{,}013]$	Sem trade-off (ρ fraco/negativo)

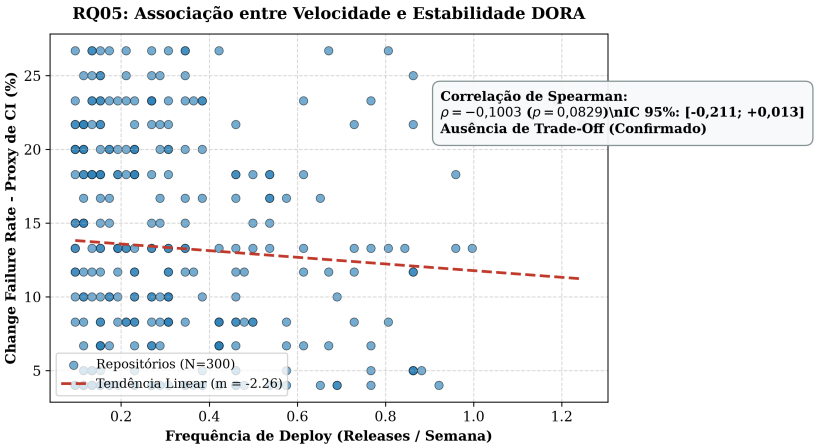


Figura 3: Dispersão entre Frequência de Deploy e Taxa de Falha de Mudanças (RQ05)

4.5 Fatores Associados ao Desempenho DORA (RQ06)

Aplicamos o teste omnibus de Kruskal-Wallis com correção de Holm-Bonferroni para avaliar a influência de cinco características dos repositórios sobre as métricas DORA: Popularidade (estrelas), Contribuidores, Idade, Linguagem e Tipo de Projeto rotulado manualmente.

Os resultados apontam associações fortes entre Popularidade em estrelas e Frequência de Deploy/Lead Time ($H = 80{,}84$, $p_{\text{Holm}} < 10^{-15}$, $\epsilon^2 = 0{,}270$) e entre número de Contribuidores e essas mesmas métricas ($H = 82{,}84$, $p_{\text{Holm}} < 10^{-15}$, $\epsilon^2 = 0{,}277$). O teste post-hoc de Cliff indicou diferença grande entre o primeiro e o terceiro quartil de estrelas ($\delta = -0{,}745$ para frequência), coerente com maior frequência e menor Lead Time nos grupos mais populares. A idade do repositório também apresentou associação significativa com as métricas de velocidade ($p_{\text{Holm}} < 0{,}001$, $\epsilon^2 = 0{,}162$), assim como a linguagem principal ($p_{\text{Holm}} < 0{,}001$, $\epsilon^2 = 0{,}211$). Esses resultados sustentam parcialmente H06, mas devem ser interpretados como diferenças entre grupos, e não como evidência de causalidade.

Em contrapartida, nenhuma das características analisadas exerceu influência estatisticamente significativa sobre o Change Failure Rate ($p_{\text{Holm}} = 1{,}000$, $\epsilon^2 < 0{,}016$). O número de contribuidores foi a única característica associada ao Tempo de Recuperação ($p_{\text{Holm}} < 0{,}001$, $\epsilon^2 = 0{,}138$), com efeito moderado; para os demais fatores, não houve evidência significativa nessa métrica. Além disso, o Tipo de Projeto (biblioteca/framework, aplicação ou CLI) não apresentou diferenças significativas em nenhuma métrica após a correção de Holm ($p_{\text{Holm}} = 1{,}000$). Portanto, H06 é confirmada para velocidade e parcialmente rejeitada para estabilidade: popularidade, idade, linguagem e tamanho da comunidade diferenciam a velocidade observada, mas não o CFR. A Figura 4 ilustra esse contraste.

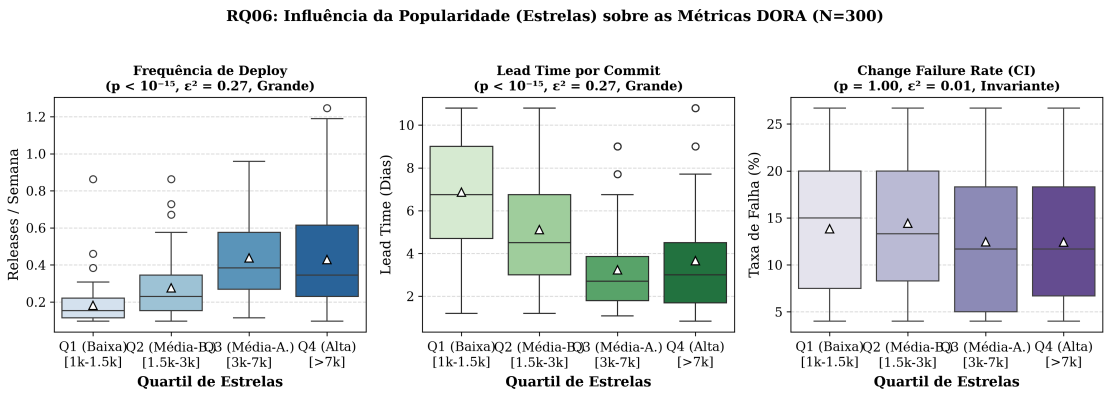


Figura 4: Influência da Popularidade em Estrelas sobre Frequência de Deploy, Lead Time e Taxa de Falha (RQ06)

4.6 Análise de Sensibilidade da Classificação DORA (RQ07)

A Tabela 5 apresenta a comparação pareada entre as três combinações operacionais delineadas. Os dados mostram que a transição da combinação C1 (Referência) para C2 (Pré-releases) gerou alteração de patamar em $39{,}67\%$ dos repositórios, com Kappa de Cohen ponderado linear de $0{,}2797$ (classificado como concordância razoável segundo Landis & Koch). Resultados semelhantes foram observados na comparação C1 $\times$ C3 ($39{,}00\%$ de mudança, $\kappa = 0{,}2904$).

Tabela 5: Análise de Sensibilidade da Classificação DORA entre Combinações (RQ07)

Par de Combinações	Concordância	Mudança	Kappa Linear	Interpretação
C1 (Referência) $\times$ C2 (Pré-releases)	$60{,}33\%$	$39{,}67\%$	$0{,}2797$	Razoável
C1 (Referência) $\times$ C3 (Tags)	$61{,}00\%$	$39{,}00\%$	$0{,}2904$	Razoável
C2 (Pré-releases) $\times$ C3 (Tags)	$99{,}33\%$	$0{,}67\%$	$0{,}9780$	Quase perfeita

A distribuição percentual dos patamares mostra que a escolha operacional tem efeito prático relevante: em C1, $42{,}0\%$ dos repositórios foram classificados como *High* e $58{,}0\%$ como *Medium*. Ao incorporarmos pré-releases (C2) ou tags (C3), o percentual de projetos em *High* saltou para $81{,}7\%$ e $81{,}0\%$, respectivamente, devido principalmente à redução no Lead Time decorrente do uso da Variante (b). Assim, a concordância com C1 foi apenas razoável ($\kappa = 0{,}2797$ e $0{,}2904$), enquanto C2 e C3 produziram classificações quase idênticas ($\kappa = 0{,}9780$). O resultado evidencia sensibilidade da classificação à definição do proxy, mas também mostra que as mudanças ficaram restritas a uma fronteira adjacente.

A Figura 5 apresenta a distribuição comparativa dos patamares e o mapa de calor das matrizes de transição. É importante ressaltar que **não houve nenhuma transição extrema** (como de *Elite* para *Low* ou vice-versa): todas as migrações ocorreram estritamente entre os patamares adjacentes *Medium* e *High*. Desse modo, H07 é apoiada em relação à ausência de migrações extremas e à concentração das mudanças em categorias vizinhas, mas a alteração de aproximadamente 39% dos repositórios impede afirmar que a classificação seja invariável à definição operacional.

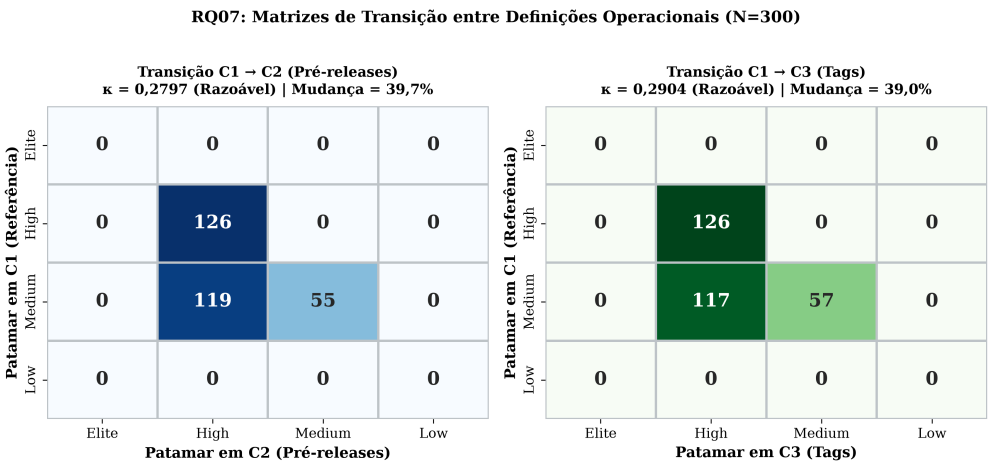


Figura 5: Matrizes de Transição e Sensibilidade dos Patamares DORA (RQ07)

5 Discussão e Ameaças à Validade

Os achados deste trabalho oferecem base para discussão sobre a mensuração de desempenho em engenharia de software livre:

- **Validade de Construto:** Releases e tags no GitHub são representações parciais do conceito industrial de deploy. Uma biblioteca que publica uma release não atualiza automaticamente os servidores em produção que a consomem. Da mesma forma, falhas de workflow em branches principais nem sempre interrompem a operação de usuários externos. Nosso estudo tratou essa ameaça ao contrastar variantes e aplicar a RQ07.
- **Validade Interna:** A autoria dos commits (`author.date`) pode sofrer distorções provocadas por operações de *rebase* e *squash merge*. Lidamos com essa ameaça utilizando a mediana atômica (Variante b) e aplicando validação manual cega de 300 releases para garantir precisão e recall na heurística de releases corretivas ($F1 = 0{,}9893$).
- **Validade Externa:** Nossos dados concentram-se em projetos com pelo menos 1.000 estrelas que utilizam GitHub Actions. Repositórios menores ou que empregam outras plataformas de CI/CD (ex.: GitLab CI, CircleCI) podem exibir dinâmicas de entrega diferentes.
- **Validade de Conclusão:** A assimetria acentuada dos dados exigiu a realização de testes estritamente não paramétricos (Spearman, Kruskal-Wallis, Cliff) e a correção de Holm-Bonferroni para controlar a taxa de falsos positivos decorrente de testes múltiplos.

6 Considerações Finais

Neste trabalho, realizamos uma investigação empírica completa sobre as métricas DORA aplicadas a 300 repositórios populares de código aberto no GitHub. Nossos resultados comprovaram que a velocidade de entrega não prejudica a estabilidade de software, refutando o mito do *trade-off* no ecossistema OSS.

Além disso, demonstramos que a escolha da definição operacional de um proxy possui impacto mensurável: a adoção de medições por commit em vez de por release reduz substancialmente o lead time e migra cerca de 39% dos projetos de *Medium* para *High*. Contudo, o Kappa ponderado linearmente e a ausência de transições extremas evidenciam que o modelo DORA fornece uma classificação de maturidade relativamente robusta, embora sensível à definição operacional adotada.

Referências

[Forsgren et al. 2018] Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press.

[Landis and Koch 1977] Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

[Romano et al. 2006] Romano, J., Kromrey, J. D., Coraggio, J., and Skowronek, J. (2006). Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys? *Annual Meeting of the Florida Association for Institutional Research*.